DATABASE MANAGEMENT SYSTEMS LAB

Lab Report Submitted to

K. R. Mangalam University

for

Bachelor of Technology

in

Computer Science and Engineering

Submitted by

YUKTI VADEHRA, ROLL NO. 2401010128

Course Teacher

Ms. Mansi Kajal

School of Engineering & Technology

EXPERIMENT-5

Aim

To create an EMPLOYEES table, insert relevant data, and perform various SQL queries to retrieve specific records using commands such as SELECT, WHERE, IN, and DISTINCT.

Exercise on retrieving records from the table-

EMPLOYEES (Employee_Id, First_Name, Last_Name, Email, Phone_Number, Hire_Date, Job_Id, Salary, Commission_Act, Manager_Id, Department_Id)

(a) Find out the employee id, names, salaries of all the employees
(b) List out the employees who works under manager 100
(c) Find the names of the employees who have a salary greater than or equal to 4800
(d) List out the employees whose last name is 'AUSTIN'
(e) Find the names of the employees who works in departments 60,70 and 80
(f) Display the unique Manager_Id.

CREATE DATABASE-

```
CREATE DATABASE practical_5_yukti;
USE practical_5_yukti;
```

Table: EMPLOYEES

Structure of Table

```
CREATE TABLE Employees (
  Employee_Id INT,
  First_Name VARCHAR(50),
  Last_Name VARCHAR(50),
  Email VARCHAR(100),
  Phone_Number VARCHAR(15),
  Hire_Date DATE,
  Job_Id VARCHAR(20),
  Salary INT,
  Commission_Act INT,
  Manager_Id INT,
  Department_Id INT
);
```

Explanation

The CREATE TABLE statement is used to define the structure of the table in the database.

- Employee_Id → Unique identifier for each employee
- First_Name, Last_Name → Store employee names
- Email, Phone_Number → Contact details
- Hire_Date → Date of joining
- Job_Id → Job role
- Salary → Salary of employee
- Commission_Act → Commission (if any)
- Manager_Id → ID of manager supervising employee
- Department_Id → Department to which employee belongs

Insert Sample Data

INSERT INTO Employees VALUES

(101, 'John', 'Smith', 'john@gmail.com', '9876543210', '2020-01-10', 'IT_PROG', 5000, NULL, 100, 60),

(102, 'Alice', 'Austin', 'alice@gmail.com', '9876543211', '2019-03-15', 'HR_REP', 4800, NULL, 100, 70),

(103, 'Bob', 'Taylor', 'bob@gmail.com', '9876543212', '2021-06-20', 'SALES', 6000, 500, 101, 80),

(104, 'Carol', 'Austin', 'carol@gmail.com', '9876543213', '2018-07-25', 'IT_PROG', 4500, NULL, 102, 60),

(105, 'David', 'Lee', 'david@gmail.com', '9876543214', '2022-02-10', 'HR_REP', 7000, NULL, 100, 90);

Explanation

The INSERT INTO statement is used to add records into the table.

- Each row represents one employee
- Values must be inserted in the same order as columns
- NULL indicates no value (e.g., no commission)

Questions

(a) Find employee id, names, and salaries

SELECT Employee_Id, First_Name, Last_Name, Salary

FROM Employees;

Employee_Id	First_Name	Last_Name	Salary
101	John	Smith	5000
102	Alice	Austin	4800
103	Bob	Taylor	6000
104	Carol	Austin	4500
105	David	Lee	7000

Explanation

- SELECT is used to retrieve specific columns
- Only required columns are selected instead of all data
- FROM Employees specifies the table

(b) List employees who work under manager 100

SELECT *

FROM Employees

WHERE Manager_Id = 100;

Employee_Id	First_Name	Last_Name	Email	Phone_Number	Hire_Date	Job_Id	Salary	Commission_Act	Manager_Id	Department_Id
101	John	Smith	john@gmail.com	9876543210	2020-01-10	IT_PROG	5000	NULL	100	60
102	Alice	Austin	alice@gmail.com	9876543211	2019-03-15	HR_REP	4800	NULL	100	70
105	David	Lee	david@gmail.com	9876543214	2022-02-10	HR_REP	7000	NULL	100	90

Explanation

* selects all columns
WHERE clause filters records based on a condition
Only employees with Manager_Id = 100 are displayed

(c) Find employees with salary ≥ 4800

SELECT First_Name, Last_Name

FROM Employees

WHERE Salary >= 4800;

	First_Name	Last_Name
▶	John	Smith
	Alice	Austin
	Bob	Taylor
	David	Lee

Explanation

- >= is a comparison operator
- Filters employees whose salary is greater than or equal to 4800
- Only names are displayed as required

(d) Last name = 'AUSTIN'

SELECT *

FROM Employees

WHERE Last_Name = 'Austin';

	Employee_Id	First_Name	Last_Name	Email	Phone_Number	Hire_Date	Job_Id	Salary	Commission_Act	Manager_Id	Department_Id
▶	102	Alice	Austin	alice@gmail.com	9876543211	2019-03-15	HR_REP	4800	NULL	100	70
	104	Carol	Austin	carol@gmail.com	9876543213	2018-07-25	IT_PROG	4500	NULL	102	60

Explanation

- String values are enclosed in single quotes
- Exact match is performed
- Returns all employees with last name Austin

(e) Employees working in departments 60, 70, 80

SELECT First_Name, Last_Name
FROM Employees
WHERE Department_Id IN (60, 70, 80);

	First_Name	Last_Name
▶	John	Smith
	Alice	Austin
	Bob	Taylor
	Carol	Austin

Explanation

- IN operator checks multiple values
- Equivalent to multiple OR conditions
- Simplifies query readability

(f) Display unique Manager_Id

SELECT DISTINCT Manager_Id

FROM Employees;

	Manager_Id
▶	100
	101
	102

Explanation

- DISTINCT removes duplicate values
- Returns only unique manager IDs
- Useful for identifying different managers

<u>Result</u>

The SQL queries were successfully executed on the **EMPLOYEES** table, and the required data was retrieved based on different conditions such as salary, manager ID, department ID, and employee names. The output displayed correct and filtered records using SQL commands like **SELECT, WHERE, IN, and DISTINCT**.